



华中科技大学

数据库系统原理实践报告

专 业:	大数据科学与技术
班 级:	BD2401
学 号:	U202414665
姓 名:	刘光悦
指导教师:	潘鹏

分数	
教师签名	

2026 年 7 月 2 日

教师评分页

子目标	子目标评分
1	
2	
3	
4	
5	

总分	
----	--

目 录

1 课程任务概述	1
2 任务实施过程与分析	2
2.1 基于火车运营系统的数据查询	3
2.1.1 第 4 关：武汉直达厦门的列车经停时刻表	3
2.1.2 第 6 关：身份证的秘密	4
2.1.3 第 12 关：各类别列车的车次数	5
2.1.4 第 13 关：福建铁路交通欠发达地区	6
2.2 存储过程与事务	7
2.2.1 第 1 关：使用流程控制语句的存储过程	8
2.2.2 第 3 关：使用事务的存储过程	9
2.3 数据库设计与实现	10
2.3.1 从概念模型到 MySQL 实现	10
2.3.2 从需求分析到逻辑模型	12
2.3.3 建模工具的使用	13
2.3.4 制约因素分析与设计	14
2.3.5 工程师责任及其分析	15
2.4 数据库应用开发(JAVA 篇)	16
2.4.1 客户修改密码	16
2.4.2 事务与转账操作	17
2.4.3 把稀疏表格转为键值对存储	18
3 课程总结	20
3.1 总体任务与完成情况	20
3.2 主要工作归纳	20
3.3 心得体会	20

1 课程任务概述

本实践课程通过一系列基于 MySQL 的实验任务，将理论知识应用于实践。课程内容涵盖了从基础的数据库对象管理到核心的系统机制，再到上层的应用开发与数据库设计。课程共分为 15 个实验模块，具体分解如下：

(一) 基础数据定义 (实验 1-2):

- 1) 实验 1 (Create): 学习使用 CREATE 语句创建数据库、表以及主键、外键、CHECK 等完整性约束。
- 2) 实验 2 (Alter): 学习使用 ALTER 语句修改已有的表结构和约束。

(二) 数据查询 (实验 3-5&15):

- 1) 实验 3 (Select): 在火车运营系统数据库背景下，进行单表、多表数据查询。
- 2) 实验 4 (DML): 掌握数据的插入、修改和删除操作。
- 3) 实验 5 (视图): 学习创建和使用视图。
- 4) 实验 15(查询处理与执行): 理解 MySQL 底层 SQL 查询完整执行流程，掌握元数据读取、DQL/DML 语句底层实现原理。

(三) 数据操作与高级对象 (实验 6-8):

- 1) 实验 6 (存储过程与事务): 学习编写存储过程并进行事务管理。
- 2) 实验 7 (触发器): 学习创建和使用触发器。
- 3) 实验 8 (自定义函数): 学习创建和使用用户自定义函数。

(四) 数据库系统核心机制 (实验 9-11&14):

- 1) 实验 9 (安全性): 实践数据库的安全性控制。
- 2) 实验 10 (并发控制): 理解事务隔离级别，处理并发事务冲突。
- 3) 实验 11 (备份与恢复): 实践数据库的逻辑备份与恢复。
- 4) 实验 14 (存储管理): 深入内核，实现缓冲池管理等底层存储组件。

(五) 数据库设计与应用开发 (实验 12-13):

- 1) 实验 12 (设计与实现): 包含从 E-R 图到 MySQL 实现，以及从需求分析到逻辑模型设计的全过程。
- 2) 实验 13 (Java 应用开发): 使用 JDBC 连接和操作数据库，完成查询、登录、增删改数据、事务处理（转账）等实际应用功能。

2 任务实施过程与分析

本次实践课程在头歌平台进行，实践任务均在平台上提交代码，所有完成的任务、关卡均通过了自动测评。如图 2.1 所示，本次实践最终完成了课程平台中的第 1~15 实验任务，仅跳过实验 15 的第 3 关，总计 146 分。下面将重点针对其中的 基于火车运营系统的数据查询（实验 3）、存储过程与事务（实验 6）、数据库设计与实现（实验 12）、Java 数据库应用开发（实验 13）四个实验中的子任务阐述其完成过程中的具体工作。

提交中	15. 查询处理与执行(Query Processing and Execution) (18分)	赵小松 2/3 2026-05-04 02:25 至 2026-07-08 23:59:00	开始学习
提交中	14. 存储管理(Storage Manager) (24分)	赵小松 4/4 2026-05-04 02:25 至 2026-07-08 23:59:00	开始学习
提交中	13. MySQL-数据库应用开发(JAVA篇) (12分)	赵小松 7/7 2026-05-04 02:25 至 2026-07-08 23:59:00	开始学习
提交中	12. MySQL-数据库设计与实现 (7分)	赵小松 3/3 2026-05-04 02:25 至 2026-07-08 23:59:00	开始学习
提交中	11. MySQL-备份+日志: 介质故障与数据库恢复 (3分)	赵小松 2/2 2026-05-04 02:25 至 2026-07-08 23:59:00	开始学习
提交中	10. MySQL - 并发控制与事务的隔离级别 (13分)	赵小松 6/6 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习
提交中	9. MySQL-安全性控制 (2分)	赵小松 2/2 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习
提交中	8. MySQL-用户自定义函数 (3分)	赵小松 1/1 2026-05-03 10:42 至 2026-07-08 23:59:00	开始
提交中	7. MySQL-触发器 (3分)	赵小松 1/1 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习
提交中	6. MySQL - 存储过程与事务 (9分)	赵小松 3/3 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习
提交中	5. MySQL-视图 (4分)	赵小松 2/2 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习
提交中	4. MySQL-数据的插入、修改与删除(Insert,Update,Delete) (7分)	赵小松 6/6 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习
提交中	3. MySQL-基于火车运营系统的数据查询(Select) (36分)	赵小松 16/16 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习
提交中	2. MySQL-表结构与完整性约束的修改(ALTER) (5分)	赵小松 4/4 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习
提交中	1. MySQL-数据库、表与完整性约束的定义(Create) (6分)	赵小松 6/6 2026-05-03 10:42 至 2026-07-08 23:59:00	开始学习

图 2.1 实验完成情况总览

2.1 基于火车运营系统的数据查询

本小节依托火车运营数据库完成多类 SELECT 查询实验，覆盖多表关联、子查询、自连接、聚合统计、字符串函数、条件分支、嵌套 EXISTS 校验等 MySQL 核心查询语法。

本关共 15 个子任务，已完成 15 个子任务。以下选出一些具有代表性的、有较强的综合性子任务来进行阐释说明，每关统一分为 关卡任务、解读要求、实现思路、关卡亮点、代码实现 五个部分来讲解。

2.1.1 第 4 关：武汉直达厦门的列车经停时刻表

(1) 关卡任务

查询同时途经武汉、厦门，且武汉停靠站序早于厦门的直达列车，筛选其中车次号最大的列车，输出该车完整经停时刻表，包含站序、站名、到站时间、离站时间，按站序升序排列。

(2) 解读要求

涉及数据表：route（车次总表）、timetable（停靠时刻表）、station（车站信息表）；

输出字段：stop_seq、station_name、arrival_time、departure_time；

核心要解决的问题：区分往返车次、筛选双向途经两站的正向列车、取编号最大车次、完整展示该车全部停靠站点。

(3) 实现思路

内层子查询通过 EXISTS 多表关联校验列车同时经过武汉与厦门，利用 stop_seq 约束武汉停靠顺序在前，对符合条件车次按 train_no 降序，LIMIT 1 提取编号最大车次；外层根据筛选出的目标车次查询全部停靠记录，最后按停靠站序升序输出。

(4) 关卡亮点

这道题综合多层嵌套子查询与 EXISTS 存在性判断，用站点序号解决双向车次混淆问题。之前只会简单单表查询，做完这道题学会用多层嵌套分步过滤复杂业务条件，对多站点路径筛选类查询有了清晰解题思路。

(5) 代码实现：

```
SELECT stop_seq,station_name,arrival_time,departure_time
FROM timetable
WHERE train_no=(
    SELECT train_no FROM route WHERE EXISTS(
```

```

SELECT 1 FROM timetable t1 JOIN station s1 ON t1.station_name=s1.sname WHERE
t1.train_no=route.train_no AND s1.city='武汉'
) AND EXISTS(
SELECT 1 FROM timetable t2 JOIN station s2 ON t2.station_name=s2.sname WHERE
t2.train_no=route.train_no AND s2.city='厦门'
AND t2.stop_seq>(SELECT MIN(t3.stop_seq) FROM timetable t3 JOIN station s3 ON
t3.station_name=s3.sname WHERE t3.train_no=route.train_no AND s3.city='武汉')
) ORDER BY train_no DESC LIMIT 1
) ORDER BY stop_seq;

```

代码 1 武汉直达厦门正向最大车次完整经停时刻表查询 SQL

2.1.2 第 6 关：身份证的秘密

(1) 关卡任务

依据身份证 18 位校验规则，查询所有身份证校验码不符合规范的乘客信息，输出乘客编号、身份证号、姓名、性别、出生日期。

(2) 解读要求

涉及数据表：passenger（乘客表）、district（行政区编码表）；

输出字段：pid、card_no、pname、gender、dob；

核心要解决的问题：拆分身份证字符、按固定加权系数计算校验值、匹配标准校验码，筛选校验码不匹配的异常证件。

(3) 实现思路

使用 LEFT、SUBSTRING、RIGHT 字符串函数拆分身份证每一位数字，结合 district 表获取前 6 位行政区加权值，按国标加权公式求和；通过 MOD 取余，搭配 ELT 函数映射标准校验码，对比身份证最后一位字符，不匹配则筛选输出对应乘客信息。

(4) 关卡亮点

题目把纸质证件校验规则完整转化为 SQL 表达式，大量用到字符串截取、数学运算、条件映射函数。让我明白生活里的校验逻辑都能用纯 SQL 实现，不再单纯认为 SQL 只能做简单数据筛选，拓展了 SQL 业务计算的使用场景。

(5) 代码实现

```

SELECT pid,card_no,pname,gender,dob
FROM passenger
WHERE RIGHT(card_no,1) != ELT(MOD(
    (SELECT v FROM district WHERE ac=LEFT(card_no,6))
+SUBSTRING(card_no,7,1)*2+SUBSTRING(card_no,8,1)*1+SUBSTRING(card_no,9,1)*6
+SUBSTRING(card_no,10,1)*3+SUBSTRING(card_no,11,1)*7+SUBSTRING(card_no,12,1)*9
+SUBSTRING(card_no,13,1)*10+SUBSTRING(card_no,14,1)*5+SUBSTRING(card_no,15,1)*8
+SUBSTRING(card_no,16,1)*4+SUBSTRING(card_no,17,1)*2,11)+1,'1','0','X','9','8','7','6','5','4','3','2');

```

代码 2 身份证校验码不符合规范的乘客信息查询 SQL

2.1.3 第 12 关：各类别列车的车次数

(1) 关卡任务

按车次首字母区分列车类型, 统计每种列车类型对应的总车次数, 输出列车类别、车次数, 自定义规则排序结果。

(2) 解读要求

涉及数据表: route (车次信息表); 输出字段: 列车类别、车次数;

核心要解决的问题: 截取车次首字母做分类、字母映射为中文列车类型、分组统计数量、自定义非字典序排序。

(3) 实现思路

子查询用 SUBSTRING 提取车次首字母, 通过 CASE 语句将 G/C/D/Z/T/K/L/Y 映射为对应列车中文名称; 基于分类字段 GROUP BY 分组, 搭配 COUNT 统计每组车次总量; 外层 ORDER BY 嵌套 CASE 设置各类列车排序权重, 实现自定义输出顺序。

(4) 关卡亮点

综合字符串函数、CASE 条件分支、分组聚合、自定义排序四类知识点。之前分组后只能按字段默认排序, 这道题学会通过自定义权重实现业务指定排序, 学会把编码字段转换成可读业务文本。

(5) 代码实现

```

SELECT
CASE first_letter
    WHEN 'G' THEN '高速动车组旅客列车'
    WHEN 'C' THEN '城际动车组旅客列车'
    WHEN 'D' THEN '动车组旅客列车'
    WHEN 'Z' THEN '直达特快旅客列车'

```



```

        WHEN 'T' THEN '特快旅客列车'
        WHEN 'K' THEN '快速旅客列车'
        WHEN 'L' THEN '临时旅客列车'
        WHEN 'Y' THEN '旅游列车'
        ELSE '其他'
    END AS 列车类别,
    COUNT(*) AS 车次数
FROM (
    SELECT SUBSTRING(train_no, 1, 1) AS first_letter
    FROM route
) AS t
GROUP BY first_letter
ORDER BY
    CASE first_letter
        WHEN 'D' THEN 1
        WHEN 'G' THEN 8
        WHEN 'C' THEN 3
        WHEN 'K' THEN 4
        WHEN 'T' THEN 5
        WHEN 'Z' THEN 6
        WHEN 'L' THEN 7
        WHEN 'Y' THEN 2
        ELSE 9
    END;
END;
```

代码 3 各类别列车的车次数信息查询 SQL

2.1.4 第 13 关：福建铁路交通欠发达地区

(1) 关卡任务

查询福建省内仅有一趟列车途经的车站

(2) 解读要求

涉及数据表：station（车站表）、district（行政区表）、timetable（停靠时刻表）；

输出字段：sname、city；

核心要解决的问题：筛选福建省站点、统计每个站点途经不重复车次数量、筛选

仅 1 趟列车停靠的站点。

(3) 实现思路

通过 EXISTS 关联 district 表限定省份为福建省，嵌套子查询统计每个站点 DISTINCT train_no 总数，添加计数等于 1 的筛选条件；最后根据车站编号 sid 升序排序输出站点信息。

(4) 关卡亮点

题目用 “途经车次数量 = 1” 定义 “交通欠发达”，把业务描述转化为聚合统计筛选条件，这个思路对我启发很大。以后遇到类似 “业务特征量化筛选” 需求，都可以先用聚合统计计算指标，再作为筛选条件过滤数据。

(5) 代码实现

```
SELECT s.sname, s.city
FROM station s
WHERE EXISTS (
    SELECT 1 FROM district d
    WHERE d.city = CONCAT(s.city, '市') AND d.prov = '福建省'
)
AND EXISTS (
    SELECT 1 FROM timetable t WHERE t.station_name = s.sname
)
AND (
    SELECT COUNT(DISTINCT t.train_no)
    FROM timetable t
    WHERE t.station_name = s.sname
) = 1
GROUP BY s.sname, s.city
ORDER BY MIN(s.sid);
```

代码 4 福建省内仅有一趟列车途经的车站查询 SQL

2.2 存储过程与事务

本关通过三个递进的编程任务，练习 MySQL 中定义和使用存储过程的核心技能，并理解事务在保证数据一致性中的关键作用。

本关共有 3 个子任务，实际完成 3 个。以下选择第 1, 3 子关进行详细阐释。

2.2.1 第 1 关：使用流程控制语句的存储过程

本关任务要求创建输入型存储过程 `sp_fibonacci`，接收整型参数 `m`，自动向 `fibonacci` 数据表插入斐波那契数列前 `m` 项数据，熟练掌握 MySQL 存储过程变量声明、分隔符修改、循环控制、批量插入等基础语法。本关完成代码如下：

```
1. use fib;
2. drop procedure if exists sp_fibonacci;
3. delimiter $$
4. truncate table fibonacci; -- 表初始化：清空原有所有数据
5. create procedure sp_fibonacci(in m int)
6. begin
7.     declare x0,x1,i,xt int; -- 声明局部变量
8.     insert into fibonacci values(0,0); -- 插入第 0 项
9.     set x0 = 0,x1 = 1,i = 1;
10.    while i < m DO
11.        xt = x0 + x1; -- 计算下一个斐波那契数
12.        insert into fibonacci values(i,x1); -- 存入当前项
13.        -- 滚动更新前后两项，为下一轮计算做准备
14.        set x0 = x1;
15.        set x1 = xt;
16.        set i = i+1; -- 计数器+1
17.    end while;
18. end $$
19. delimiter ;
```

代码 5 使用流程控制语句计算斐波那契数列的存储过程 SQL

核心逻辑说明

1. 环境兼容处理

先删除同名存储过程、清空数据表，避免重复执行报错；使用 `DELIMITER $$` 临时修改语句结束符，防止存储过程内部分号提前终止定义，完成后恢复默认分号。

2. 变量与初始项

通过 `DECLARE` 在复合语句块内定义局部变量，分别存储前一项、当前项、计

数器、临时和；固定插入第 0 项 (序号 0, 数值 0) 作为数列基准。

3. WHILE 循环迭代计算

循环条件 $i < m$ 控制生成总条数；每次迭代先计算下一项斐波那契值，写入当前序号与数值，再滚动更新前后两项数值、计数器自增，实现迭代递推。

4. 功能优势

将数列生成、批量插入逻辑封装，一次调用即可完成批量数据写入，复用性强，简化重复 SQL 编写，为后续带事务、带校验的复杂存储过程打下基础。

2.2.2 第 3 关：使用事务的存储过程

本关任务要求编写存储过程 `sp_transfer`，用于实现在两个银行账户之间进行转账。此过程需处理复杂的业务规则，并确保操作的原子性，即转账要么完全成功，要么完全失败，数据库状态不会处于中间状态。流程示意图 2.2

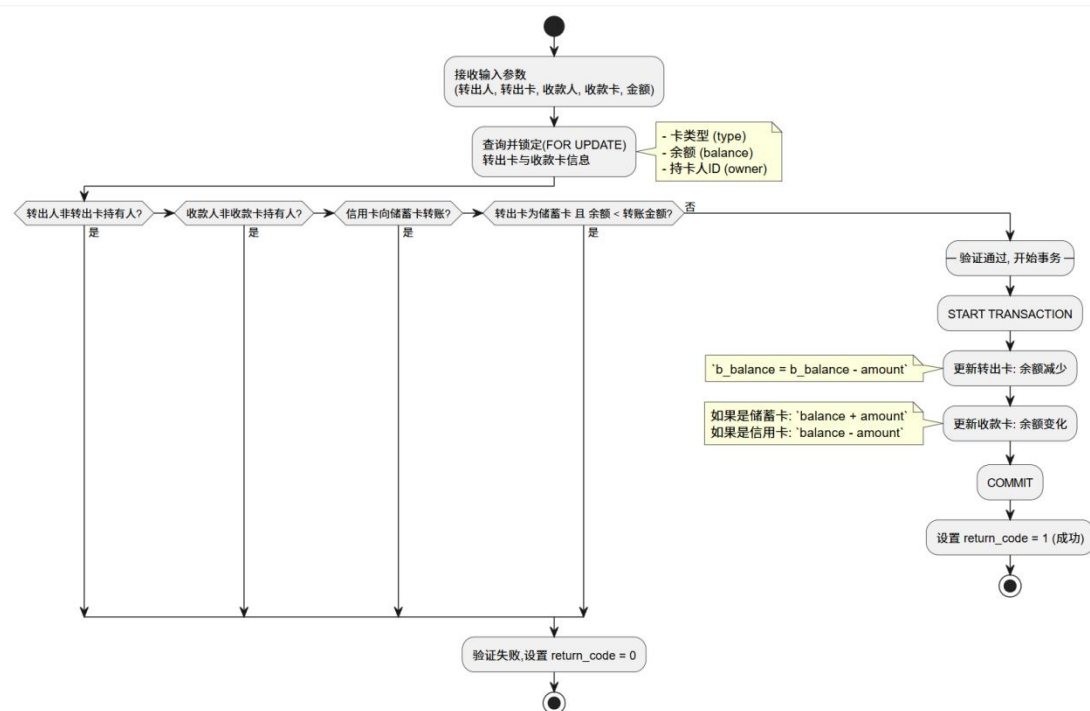


图 2.2 使用事务的存储过程流程图

存储过程 `sp_transfer` 接收五个输入 (IN) 参数，包括付款人、收款人、双方卡号及转账金额。输出 (OUT) 参数 `return_code` 用于向调用者反馈操作结果 (1 为成功，0 为失败)。

执行更新操作之前, 过程首先使用 `SELECT ... INTO ...` 语句查询转出卡和转入卡的信息。这里的关键点在于查询语句末尾使用 `FOR UPDATE` 子句, 它会对查询结果中的相应行施加一个排他锁, 防止在本次转账事务完成之前有其他并发事务修改这两张卡的余额, 从而可以避免“脏读”和“不可重复读”等并发问题, 保证了数据的一致性。

在启动事务前, 还需进行一系列严格的条件检查, 提前拦截无效操作, 减少不必要的事务开销。首先, 进行所有权验证, 检查发起转账的客户是否是转出卡的合法持有人, 以及收款人是否是收款卡的持有人。然后, 进行转账规则验证, 禁止从“信用卡”向“储蓄卡”转账, 且确保“储蓄卡”作为转出方时, 其余额充足。任何一条规则不满足, 都会立即将 `return_code` 设为 0, 并终止执行。

在所有前置校验通过后, 才通过 `START TRANSACTION` 显式地开启一个事务。所有后续的数据库更新操作将作为一个不可分割的单元执行。根据卡的类型更新双方账户余额。对于储蓄卡是简单的加减法。对于信用卡还款 (向信用卡转账), 余额是做减法, 因为其 `b_balance` 字段代表的是已透支金额。当所有更新语句成功执行后, 使用 `COMMIT` 提交事务, 使所有更改永久生效。同时设置 `return_code` 为 1, 表示成功。

2.3 数据库设计与实现

本关主要围绕数据库设计与实现的完整流程展开, 主要包含两大模块:

一是从需求到模型, 基于给定的影院管理系统业务场景, 进行需求分析, 绘制 E-R 图完成概念与逻辑建模, 最终产出规范化的关系模式。

二是从模型到实现, 将一个已定义好的逻辑模型转化为 MySQL 上的物理数据库。

本关共有 3 个子关卡, 实际完成 3 个子关卡。以下将详细阐述这 3 关。

2.3.1 从概念模型到 MySQL 实现

本关要求根据提供的关于机票订票系统逻辑模型的 E-R 图 (见图 2.3) 及文字描述, 编写一套完整的 SQL DDL 脚本。该脚本需在 MySQL 中创建名为 `flight_booking` 的数据库, 并精确构建 `user`, `passenger`, `airport`, `airline`, `airplane`,

flightschedule, flight, ticket 等八个表，同时定义各表的主键、外键、非空、唯一、默认值以及索引等约束，从而将抽象的逻辑设计转化为具体的物理数据库实现。

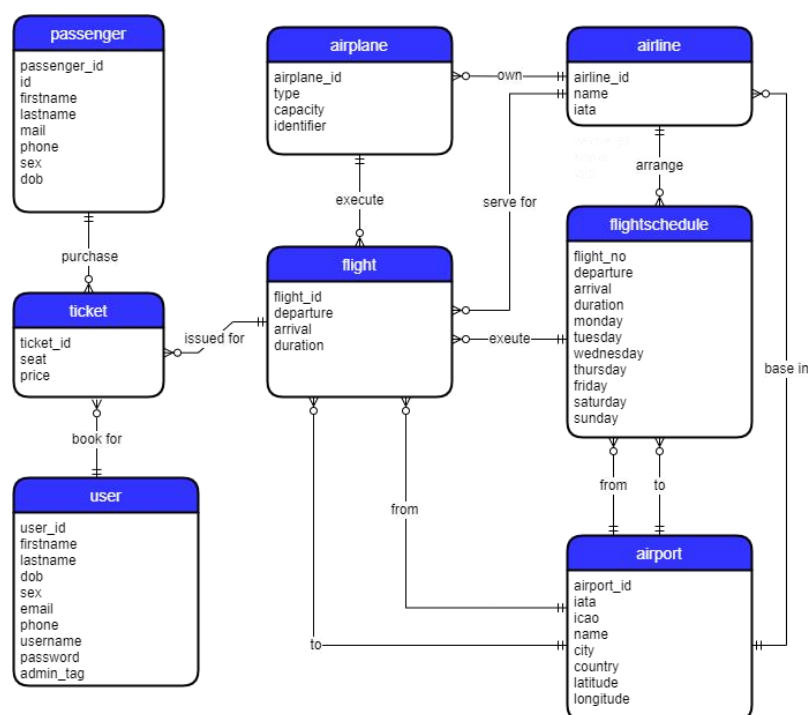


图 2.3 机票订票系统概念模型 ER 图

本关的核心在于：将 E-R 图中定义的实体、属性和关系，逐一映射为 SQL 中的表、列和约束。整体思路遵循“先独立实体，后关联实体”的原则，以确保在定义外键时，其引用的主键已经存在。

1) 数据库初始化

为保证脚本的可重复执行性，在脚本开头使用 DROP DATABASE IF EXISTS flight_booking;和 CREATE DATABASE flight_booking;，确保每次运行都能在一个干净的环境中进行。随后使用 USE flight_booking;指定后续操作的上下文。

2) 表结构创建

严格遵循“先独立实体，后关联实体”的创建顺序。通过梳理 E-R 图中的依赖关系，我们可以规划出一条无冲突的创建路径：user, passenger, airport -> airline -> airplane, flightschedule -> flight -> ticket。

首先，创建独立实体表 user, passenger, airport。这些表的创建相对直接，主要是将模型中的属性精确转换为 MySQL 的列定义，并根据文字描述中的约束需求，附加 PRIMARY KEY AUTO_INCREMENT, NOT NULL, UNIQUE, DEFAULT 等约束。

然后, 依次创建关联实体表 `airline -> airplane`, `flightschedule -> flight -> ticket`。在 `airline` 表中, 还根据要求为 `name` 创建索引, 提升查询性能。

需要注意的是, 在 `flightschedule` 和 `flight` 表中, 表示出发地和到达地的外键列被命名为 `from` 和 `to`。这两个词是 SQL 的保留关键字。为了避免语法错误, 需要使用反引号 (``) 将它们括起来, 如 ``from`` 和 ``to``。

由于篇幅限制, 最终实现的 SQL 代码详见源代码附件。

2.3.2 从需求分析到逻辑模型

本关任务要求根据影院管理系统的业务需求, 先绘制 E-R 图, 再将其转换为关系模式。

1) E-R 图绘制

首先, 我们需要仔细分析本关任务描述中的业务功能, 识别出核心的实体和它们之间的关系。根据描述, 可以明确地识别出五个实体: 电影(movie)、顾客(customer)、放映厅(hall)、排场(schedule) 和 电影票(ticket)。每个实体的属性也已在描述中清晰列出。其中, 顾客与电影票是“购买”关系, 一个顾客可购买多张票, 一张票只被一个顾客购买, 是 1:N 关系; 电影票与排场是“属于”关系, 多张票属于一个排场, 是 N:1 关系; 排场与电影是“放映”关系, 一个排场放映一部电影, 一部电影可有多个排场, 是 N:1 关系; 排场与放映厅是“位于”关系, 一个排场位于一个放映厅, 一个放映厅可有多个排场, 是 N:1 关系。

基于以上分析, 使用 `tikz` 工具绘制出 E-R 图, 见图 2.4。

2) 转换关系模式

有以上分析可知, 各实体间均为 1:N 关系, 而无多对多关系, 因此无须为关系本身创建新的、独立的关联表。所有的联系都可以通过在“N”端实体对应的关系模式中, 添加“1”端实体的主码作为外码来实现。

因此, 每个实体集都转换为一个独立的关系模式。实体的属性成为关系模式的属性, 实体的标识符成为关系模式的主码。最终的关系模式由代码 6 呈现。

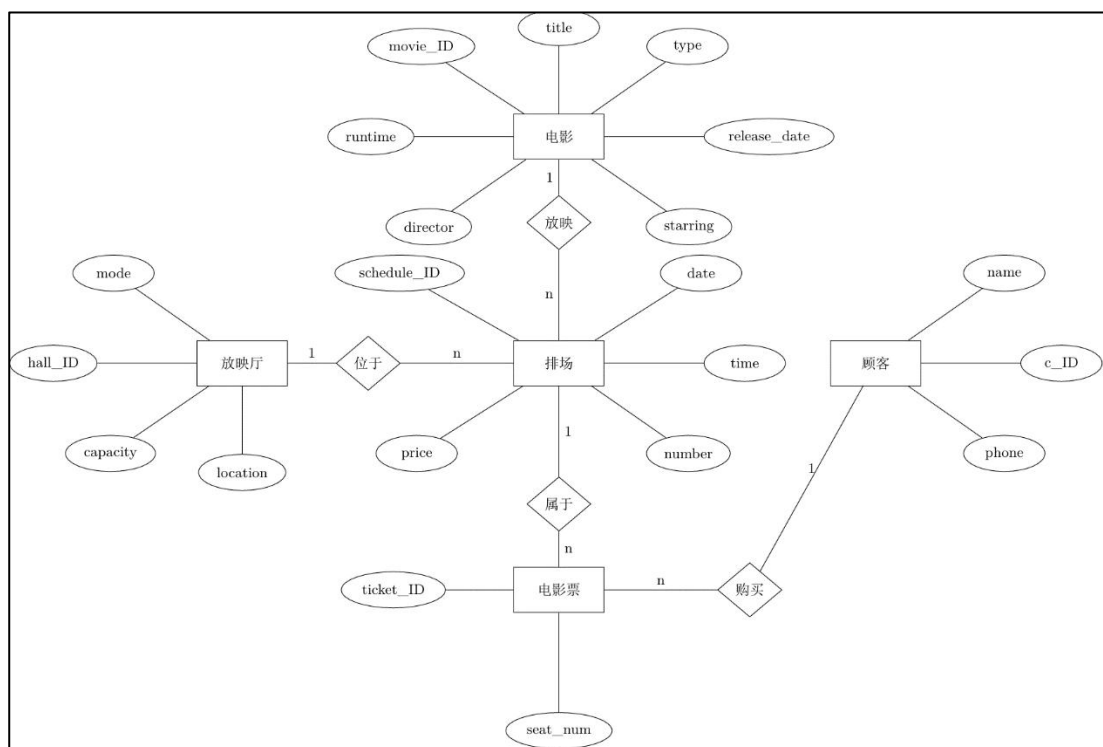


图 2.4 影院管理系统 E-R 图模型

1. movie(movie_ID, title, type, runtime, release_date, director, starring), 主码: (movie_ID)
2. customer(c_ID, name, phone), 主码: (c_ID)
3. hall(hall_ID, mode, capacity, location), 主码: (hall_ID)
4. schedule(schedule_ID, date, time, price, number, movie_ID, hall_ID), 主码: (schedule_ID);
外码: (movie_ID)→movie(movie_ID), (hall_ID)→hall(hall_ID)
5. ticket(ticket_ID, seat_num, schedule_ID, c_ID), 主码: (ticket_ID);
外码: (schedule_ID)→schedule(schedule_ID), (c_ID)→customer(c_ID)

代码 6 关系模式

2.3.3 建模工具的使用

本关任务旨在了解使用专业的数据库建模工具（如 MySQL Workbench）进行数据库设计，并通过其“Forward Engineering”功能，从可视化模型自动生成 SQL DDL 脚本。

第一步，如图 2.5，使用 MySQL Workbench 打开 rbac.mwb 模型文件 (File -> Open Model..->在文件浏览器窗口选中 rbac.mwb 文件)。

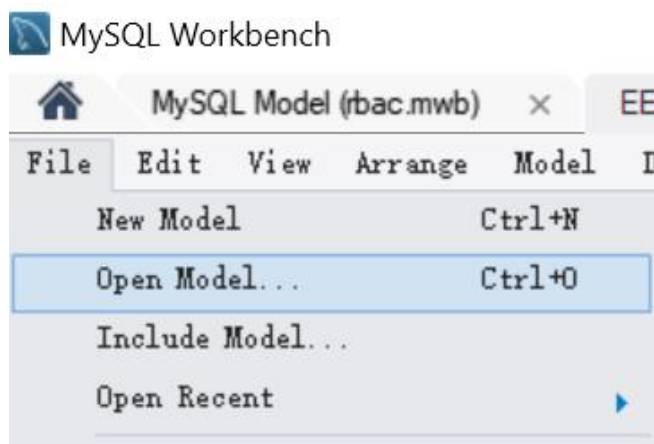


图 2.5 第一步

第二步，打开后，点击菜单上的 Database-> Forward Engineering, 如图 2.6。点击“Forward Engineering”，在弹出的导引窗口中一直点 Next, 直至 Review SQL Script 步骤，便可见到自动生成的 SQL 脚本。

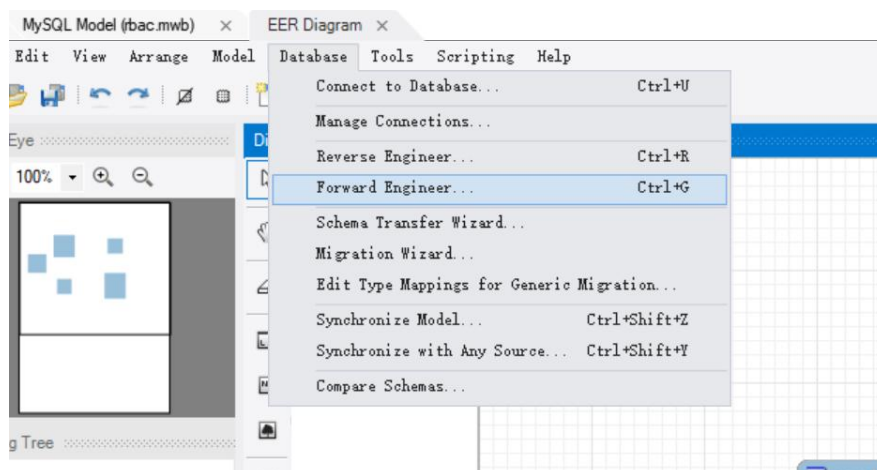


图 2.6 第二步

2.3.4 制约因素分析与设计

在数据库设计与实现的实验任务中，尽管我们的核心任务是技术实现，但在方案设计层面，我们必须将社会、安全、法律等现实世界的制约因素纳入考量。一个成功的系统不仅功能完备，更应是负责任、安全可靠且合规的。以下是在设计过程中，对这些因素的一些考虑。

1) 数据隐私

系统中收集了大量个人身份信息，例如，user 和 passenger 表中的姓名、身

身份证号、电话、邮箱、生日等。在设计上，通过 UNIQUE 约束保证了身份证、手机号等信息的唯一性，这既是业务要求，也符合数据管理的准确性原则；在实际应用中，对这些敏感数据的存储、传输和访问必须进行加密和严格的权限控制，以遵守有关的法律法规。

2) 文化差异

user 表中设置 firstname 和 lastname 字段，而非单独的 name 字段，是为了适应东西方对姓名的不同表达习惯。airport 表中设置 country 和 city 字段，也是为了适应国际化需求。

3) 访问安全与权限控制

在第一关机票预订系统的 user 表中，通过 admin_tag 字段明确区分了普通用户和管理员。第 3 关的 RBAC 模型通过将权限 (apright) 赋予角色 (aprole)，再将角色分配给用户 (apgroup)，遵循了“最小权限原则”，确保用户只能访问其职责所需的数据和功能，有效防止了越权操作和内部数据泄露，保障了系统的整体安全。

2.3.5 工程师责任及其分析

在设计和实现机票预订及权限管理系统的过程中，我们必须认识到，每一行代码、每一个数据表的设计，都可能与社会、安全、法律及文化等因素产生复杂的相互影响。

1) 安全与法律责任

在机票预订系统中，我们处理的是包括姓名、身份证号、联系方式在内的高度敏感个人信息。若数据库设计不周，如密码明文存储、缺少访问控制，可能导致大规模数据泄露，不仅会给旅客带来财产损失和隐私侵害，更可能触犯有关法律法规，使企业和个人面临严重的法律后果。

2) 社会与文化责任

软件系统是社会运行的工具，其设计应体现对社会多样性和人文需求的关怀。本次设计中将 user（用户）与 passenger（旅客）分离，是考虑到了现实社会中的互助行为，使系统更具人情味和实用性。

2.4 数据库应用开发(JAVA 篇)

本关旨在通过一系列递进的编程任务，锻炼使用 Java JDBC (Java DataBase Connectivity,java 数据库连接) 进行数据库应用开发的核心技能。该实验共包含七个关卡，内容涵盖了数据库的增删改查 (CRUD) 操作、用户身份验证、事务处理以及数据结构转换。

本关共有 7 个子关卡，实际完成 7 个子关卡。以下是对其中代表性的关卡的详细阐述。

2.4.1 客户修改密码

本关任务要求实现用于修改客户的登录密码的 `passwd` 方法。该方法需要接收数据库连接、客户邮箱、旧密码和新密码作为参数。在执行修改前，必须先验证客户邮箱是否存在以及旧密码是否正确。根据执行结果，方法需返回一个特定的整数值来表示不同状态：1 代表成功，2 代表用户不存在，3 代表密码错误，-1 代表程序异常。

本关的关键逻辑是“查询-验证-更新”，为保证数据操作的准确性和安全性，不能直接执行 UPDATE。

本关执行流程见示意图 2.7。流程说明如下：

首先，查询用户并验证身份，根据传入的客户邮箱 (mail) 参数查询 client 表。如果查询结果为空，说明该邮箱对应的用户不存在，方法应返回 2。如果用户存在，则从查询结果中取出该用户存储在数据库中的密码。将取出的密码与用户传入的旧密码 (password) 进行比对，如果密码不匹配，说明旧密码错误，方法应返回 3。如果旧密码验证通过，则说明身份正确。

然后，更新密码。执行 UPDATE 语句，将该邮箱对应用户的密码更新为新密码 (newPass)。根据 UPDATE 操作影响的行数判断是否更新成功。若影响行数大于 0，则表示修改成功，方法返回 1。

在整个数据库操作过程中，使用 try-catch 块捕获可能发生的 SQLException。一旦捕获到异常，说明数据库连接或 SQL 执行出现问题，方法应返回 -1。

出于安全性考虑，代码全部使用了 PreparedStatement，通过参数占位符?来设置查询和更新的条件值，而非使用 Statement，避免 SQL 注入攻击的风险。

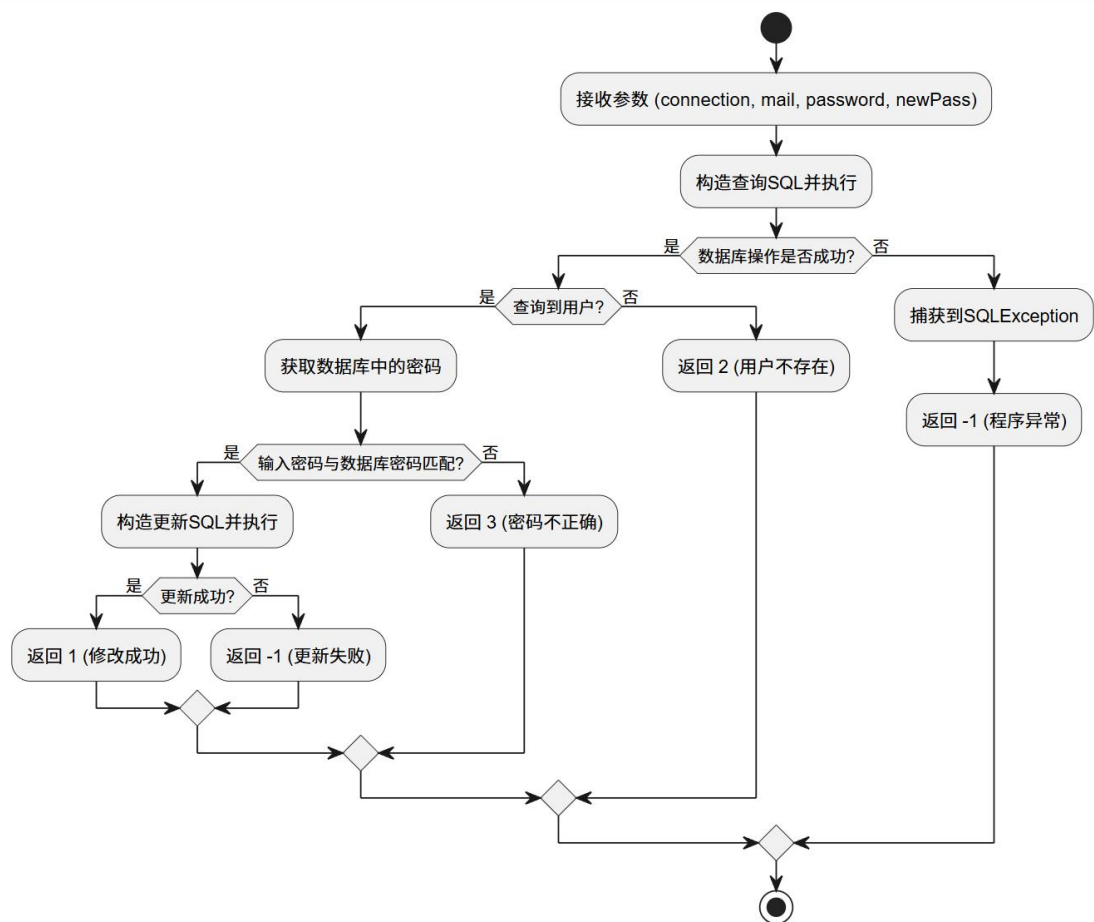


图 2.7 passwd 方法流程图

2.4.2 事务与转账操作

本关任务要求实现银行卡转账方法 `transferBalance`，模拟完整的转账流程：从一个储蓄卡账户（转出方）扣除指定金额，并将该金额增加到另一个账户（转入方）。

本关关键是利用 JDBC 的事务处理机制，确保转账操作的原子性，即转出账户的扣款和转入账户的存款必须同时成功或同时失败。转账失败的条件包括：转出或转入卡号不存在、转出卡为信用卡、或转出卡余额不足。流程说明如下：

1) 关闭事务自动提交

在所有数据库操作开始前，调用 `connection.setAutoCommit(false)` 关闭自动提交。

2) 校验转出方及转入方

查询转出卡信息，检查卡是否存在、是否为储蓄卡、以及余额是否足够支付

转账金额。任一条件不满足，则调用 `connection.rollback()` 回滚事务并返回 `false`。
查询转入卡信息，检查卡是否存在。若不存在，同样回滚事务并返回 `false`。

3) 执行转账

执行 `UPDATE` 语句，从转出卡余额中减去转账金额。根据转入卡的类型，若为储蓄卡，则余额 $b_balance = b_balance + amount$ ；若为信用卡，则代表还款，透支额 $b_balance = b_balance - amount$ 。

4) 提交或回滚

如果所有 `UPDATE` 操作都成功执行（影响行数均为 1），则调用 `connection.commit()` 提交事务，使所有更改永久生效，并返回 `true`。

如果在任何步骤中出现 SQL 异常或更新影响的行数不为 1，则在 `catch` 块中调用 `connection.rollback()` 撤销该事务内所有已执行的操作，并返回 `false`。

2.4.3 把稀疏表格转为键值对存储

本关任务要求实现一个 Java 方法，将一个多列的、数据稀疏的“宽表” (`entrance_exam`) 中的数据，转换并存储到“键-值”对形式的“长表” (`sc`) 中。

具体来说，需要读取 `entrance_exam` 表中的每一行记录，并将其中的非空成绩（如语文、数学、英语等）作为独立的行插入到 `sc` 表中，从而实现数据结构的优化，节省存储空间并便于后续的分析。

本关的关键在于将表的列转换为行，需要对查询结果进行二次处理和循环插入。执行流程图见图 2.8。流程说明如下：

首先，查询源数据，获取源表中的所有学生及其各科成绩记录，结果存放在 `rs(ResultSet)` 中。

然后，使用 `while (rs.next())` 循环遍历查询结果中的每一行，即每一个学生。对于每个学生，使用 `for` 循环遍历预先定义的科目名称数组。通过 `rs.getString(科目名)`，获取当前学生在当前科目下的成绩。

接下来，关键步骤是判断该成绩值是否为 `null`。如果成绩值不为 `null`，则调用独立的 `insertSC` 方法，将学生编号 (`sno`)、科目名 (`col`) 和成绩值 (`value`) 作为一条新记录插入到目标表 `sc` 中，若成绩值为 `null` 则跳过，继续遍历下一科目。

当两层循环结束，则所有数据转换完成。如果在此过程中的任何步骤发生错误（如数据库连接失败、SQL 语法错误等），catch 块会捕获这个异常并将其信息打印到控制台。

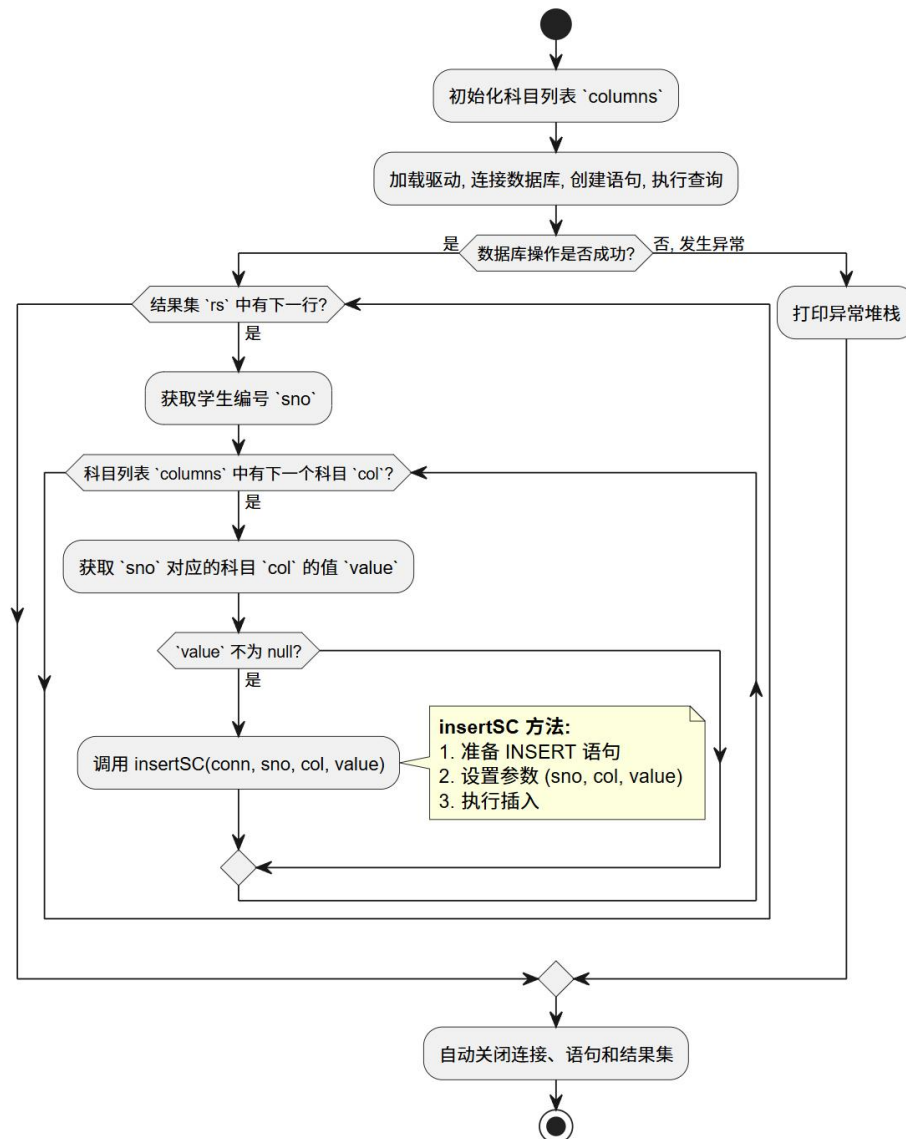


图 2.8 稀疏表格转储流程图

3 课程总结

3.1 总体任务与完成情况

本次《数据库系统原理实践》课程以 MySQL 为平台，设置了 15 个实验模块，涵盖从基础 SQL 编程、高级对象开发、并发控制与安全性管理，到数据库建模设计以及 Java 应用开发的完整技术链条。

在为期数周的实践中，我共计完成实验 1 至实验 15 的全部模块，仅跳过实验 15 的第 3 关，累计获得 146 分。

3.2 主要工作归纳

在数据定义与查询方面，我最大的收获是学会了如何将业务需求转化为规范的 SQL 语句。以火车运营系统为实践用例，面对给出的查询任务，学会理解需求，拆解任务（涉及到的数据表、表间关联条件、期望的输出结果等），最后应用专业知识将逻辑内容转化为 sql 语言，并且对一些特殊技巧和关键字的运用有了更熟练地掌握。

在高级编程与核心机制方面，我主要实践了存储过程的编写与调试。编写存储过程与普通 SQL 查询最大的不同在于需要考虑完整的执行流程和异常情况，每一步都需要编写独立的条件判断和相应的错误处理逻辑。在并发控制实验中，我通过设置不同的隔离级别，实际观察到了脏读和不可重复读的发生与消除过程，对理论课上抽象的隔离性概念有了直观理解。

在数据库设计与应用开发方面，我依据影院管理系统和机票预订系统的业务需求，完成了从需求分析、实体识别、E-R 图绘制到关系模式转换的完整设计流程，并在 Java 应用开发中掌握了一些标准操作与事务的手动控制，确保实际事务原子性的方法等等，这是非常宏观视角又兼具实用意义的一个部分，让我亲身体会到如何将所学用于解决现实问题。

3.3 心得体会

此次实践让我感触最深的有三点。

其一，课程覆盖面广、编排用心，让我获得了系统性的认知。从最基础的 SQL 语句逐步深入到存储管理底层，整个过程中我不断感受到能力的积累和提升。特别是一些启发性的题目，让我从机械地执行 SQL 转变为主动思考如何解决问题。以“福建铁路交通欠发达地区”一关为例，将该需求解读为找出“仅有一趟列车途经的车站”，这个过程让我意识到自己开始学会将模糊的业务描述转化为可计算的逻辑，将问题一步步拆解直至找到解决方案。这种从“做题”到“解决问题”的转变，是这门课最珍贵的收获之一。

其二，老师们始终关注我们的学习进度，在课上课下给予帮助支持。验收时耐心听取思路，鼓励引导，遇到卡点总能给予启发式引导而非直接给出答案，这既锻炼了独立排查问题的能力，也让我面对困难时更有信心。

其三，实验报告模板的要求极为细致——章节层级、插图尺寸、字体字号，每项都有明确规范。起初觉得繁琐，完成之后才意识到规范的工程文档本身就是专业素养的一部分，这些细节上的磨练同样让我受益匪浅。

最后，衷心感谢课程组全体老师的辛勤付出，是你们的专业与耐心，让我们得以在一次次实践中扎实成长。